

Recruitment Management System — Deliverable Report

Candidate Pipeline Tracker & Natural Language Search Platform

Project Links & Deliverables:

- GitHub: <https://github.com/ayushprajapati3002/RecruitmentSoftware>
- Live Frontend Demo: recruitment-software.vercel.app
- Live Backend API: recruitment-backend.onrender.com
- AI Chat Logs: [AI_CHAT_LOGS/README.md](#)

1. Summary of Architecture

The system is built as a modular client-server web application designed for high responsiveness, pipeline traceability, and fast conversational recruiter querying:

- **Frontend (React 19 + TypeScript + Vite):** Provides an interactive candidate recruitment pipeline (Stages: *Applied* → *Screening* → *Interview* → *Offer* → *Hired*) in Kanban and List views. Features zero-lag optimistic UI updates, dynamic contextual suggestions, and direct candidate contact displays. Styled with clean Vanilla CSS (zero Tailwind bloat).
- **Backend (Express 5 + Node.js):** RESTful API server with dedicated controllers for candidate CRUD, stage movement (advance/reject/archive), and search endpoints. Express 5 compatible CORS with credentials and Multer for resume storage.
- **Database (PostgreSQL + Prisma ORM + pg_trgm):** Stores candidates with normalized search fields and an append-only, immutable `candidate_history` audit table. Native PostgreSQL trigram indexes (`pg_trgm`) handle fuzzy text matching and typo tolerance (e.g. *Priya Sharam* → *Priya Sharma*).
- **Natural Language Processing (Groq Cloud):** Translates recruiter queries like "*candidates in screening with 3+ years experience*" into deterministic JSON filter structures in under 250ms without hallucinating database rows.

2. How to Run the Application

1. **Prerequisites:** Node.js (v18+ or v24+), PostgreSQL database instance with `pg_trgm` , and a Groq API key.
2. **Backend Setup:**

```
cd backend
npm install
# Configure backend/.env with DATABASE_URL, DIRECT_URL, and GROQ_API_KEY
npx prisma db push
npm run dev      # Runs on http://localhost:5000
```

3. **Frontend Setup:**

```
cd frontend
npm install
npm run dev      # Runs on http://localhost:5173
```

3. Key Decisions Made and Why

- **Pure Intent Parsing via Groq over Regex:** We chose high-speed cloud LLM inference over brittle regex parsing or local heavy Python NLP models. It extracts complex recruiter intents accurately with sub-250ms latency and outputs strict JSON without runtime overhead.
- **PostgreSQL pg_trgm Fuzzy Search:** Typo tolerance is resolved natively at the database index layer rather than via external vector databases or brittle hardcoded synonym maps.
- **Optimistic UI Updates:** Stage movements update state immediately in React, closing modals with zero perceived delay while syncing with the server in the background.
- **Immutable CandidateHistory Table:** Instead of overwriting a single status column, every candidate movement (advance, reject, create) creates an append-only audit record with `onDelete: NoAction`. This guarantees a verifiable compliance history.
- **Soft Deletes (isArchived):** Candidates are never permanently purged from the database. Archiving retains full candidate profile history and past hiring evaluations for future openings.
- **Vanilla CSS Design System:** Custom CSS grid and flexbox without large UI libraries ensures complete layout control across all device viewports, especially narrow phones and foldables.

4. AI Collaboration: Where I Disagreed with the AI

The Incident: Rule-Based Offline Regex Parsing vs. Pure LLM Intent Extraction

During the implementation of the natural language search feature, the AI proposed a two-step parsing strategy starting with "Step 1: Offline Rule-Based NLP". The AI wrote regular expression heuristics (e.g., `/(applied|screening|interview|offer|hired)/i`, `/(\d+)\+?\s*years?/i`) to intercept queries before passing them to the model.

My Directive: "Don't want this Step 1: Offline Rule-Based NLP. Remove the changes of step 1 only."

Why I Disagreed:

- **False Positives:** Natural conversational recruiter queries (such as *"candidates who passed screening for backend developer"*) easily trick simple regex patterns, confusing notes or job descriptions with pipeline stages.
- **Duplicate Maintenance:** Having both a rigid regex ruleset and an LLM pipeline introduces duplicate parsing logic and edge cases.
- **Speed:** Because Groq executes model inference in under 250ms, a fragile offline regex fallback was completely unnecessary. I insisted on a single, clean pipeline where the LLM produces a strict JSON schema and PostgreSQL performs retrieval and fuzzy matching.

5. What I'd Do With More Time

- **Semantic Vector Search (pgvector):** Add vector embeddings over candidate resumes and interview notes to allow conceptual queries like *"find candidates with leadership experience in cloud migrations"*.
- **Automated Resume OCR & Parsing:** Automatically parse uploaded PDF/Word resumes to auto-fill skills, work history, and contact information.
- **Interview Scheduling Integration:** Integrate Google Calendar / Outlook APIs to let recruiters send interview invitations and view open slots directly from the candidate card.
- **Real-time Multi-Recruiter Sync:** Use WebSockets or Server-Sent Events (SSE) so stage changes and notes update live for all recruiters viewing the board.
- **Role-Based Access Control (RBAC):** Add distinct permissions for Admins, Hiring Managers, and Interviewers.